Trabajo en Clase N°1

Arquitectura y Evolución del Desarrollo Móvil

Aplicaciones Móviles

Semestre 2026 A

Albán Lucas Dorian Joel

Alpusig Guaman Erick Medardo

Arcos Constante Eduardo Emmanuel

Escuela Politécnica Nacional

15 de abr. de 2026

Tabla de contenido

1. Introducción 2
2. Desarrollo 2
2.1. Parte 1: El Viaje Arquitectónico del Desarrollo Móvil 2
2.1.1. Desarrollo Nativo Puro (iOS/Android) 2
2.1.2. La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic) 3
2.1.3. El Puente Nativo-JavaScript (React Native) 3
2.1.4. El Motor de Renderizado Propio (Flutter) 4
2.1.5. El Futuro: Foldables, AR y Wearables 4
2.2. Parte 2: Análisis Crítico de Aplicaciones (Tinder y Waze) 5
2.2.1. Ingeniería Inversa de la Decisión 5
2.2.2. Análisis de Rendimiento y Arquitectura 6
2.2.3. El Desafío de las Nuevas Pantallas 6
2.2.4. Veredicto Crítico: ¿Qué elegir hoy? 6
2.2.5. Pregunta Final: ¿Qué es un aplicativo nativo y cómo se clasifican? 7

1. Introducción

En la actualidad, el desarrollo de aplicaciones móviles ha trascendido la simple escritura de código para convertirse en un campo complejo de decisiones arquitectónicas estratégicas. Para los ingenieros de la Facultad de Sistemas, comprender la infraestructura subyacente de las herramientas móviles es vital, ya que estas determinan atributos de calidad críticos como el rendimiento, la escalabilidad y el costo de mantenimiento.

El presente informe analiza la evolución del desarrollo móvil, desde el purismo del desarrollo nativo hasta el surgimiento de motores de renderizado propio y arquitecturas basadas en puentes. A través de un análisis comparativo de casos de éxito como Tinder y Waze, se explorará cómo las demandas de interactividad, procesamiento en tiempo real y la llegada de nuevos paradigmas como las pantallas plegables y la Realidad Aumentada (AR) condicionan la elección de una tecnología sobre otra. El objetivo es fundamentar, bajo criterios estrictos de ingeniería, por qué el "vanguardismo tecnológico" debe estar siempre supeditado a la eficiencia arquitectónica.

2. Desarrollo

2.1. Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

2.1.1. Desarrollo Nativo Puro (iOS/Android)

Tecnologías: Swift (iOS), Kotlin/Java (Android).

Funcionamiento: Se comunica directamente con el Kernel del SO y utiliza las APIs propietarias. El código se compila a binario (instrucciones de CPU).

Análisis de Relevancia: Vigente y esencial. Sigue siendo la única opción para aplicaciones que requieren acceso de bajísima latencia al hardware (sensores, cámara, Bluetooth stack) o que necesitan exprimir al máximo la GPU/CPU.

2.1.2. La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic)

Tecnologías: HTML, CSS, JavaScript.

Funcionamiento: La app es un contenedor (browser) que renderiza una web. El acceso al hardware se hace mediante un "Bridge" que traduce llamadas de JS a código nativo.

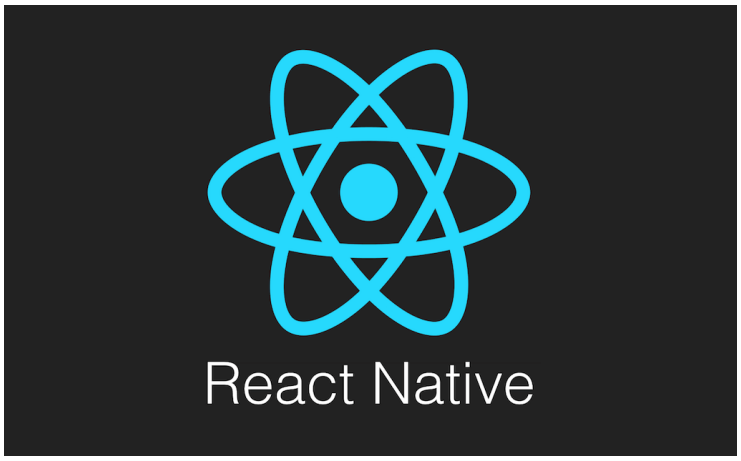
Análisis de Relevancia: En declive. El problema es el **DOM (Document Object Model)**. La manipulación del DOM en dispositivos móviles es costosa y genera el famoso "lag" al hacer scroll o animaciones. Solo es relevante para aplicaciones empresariales internas de baja interactividad.

2.1.3. El Puente Nativo-JavaScript (React Native)

Tecnologías: JavaScript/TypeScript.

Funcionamiento: La lógica corre en JS, pero la UI es nativa. Utiliza un **Bridge** (puente) que serializa mensajes en JSON de forma asíncrona para decirle al SO qué dibujar.

Análisis de Relevancia: Muy relevante. Permite compartir lógica de negocio manteniendo componentes visuales nativos. Su mayor debilidad es la saturación del puente si hay demasiados mensajes (como en una animación compleja).



2.1.4. El Motor de Renderizado Propio (Flutter)

Tecnologías: Dart, Motor Skia/Impeller.

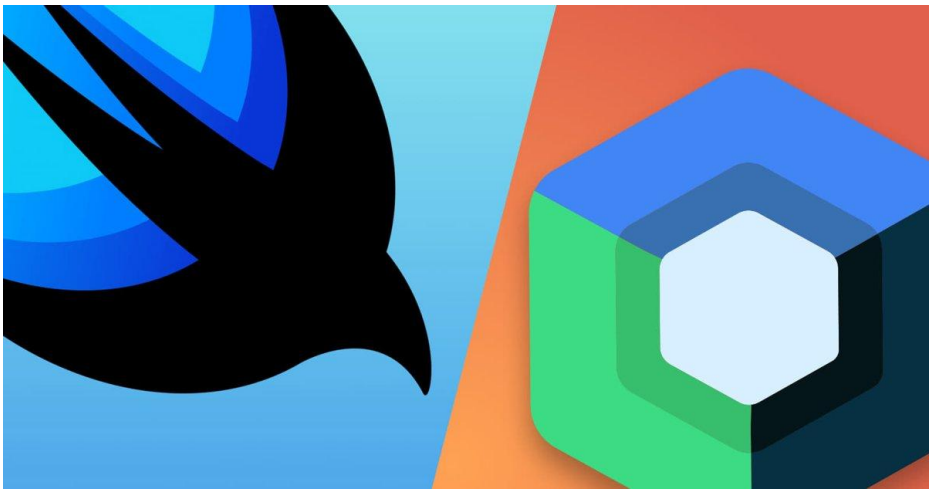
Funcionamiento: No usa componentes del SO. Flutter toma el control de cada píxel de la pantalla y lo dibuja directamente usando la GPU, de forma similar a un motor de videojuegos.

Análisis de Relevancia: Líder actual en cross-platform. Ofrece 60/120 FPS constantes y una consistencia visual perfecta entre marcas de teléfonos.

2.1.5. El Futuro: Foldables, AR y Wearables

Foldables: Requiere arquitecturas basadas en **Layouts Dinámicos** y gestión de estado reactiva (como Jetpack Compose o SwiftUI) para detectar el cambio de Hinge (bisagra) en tiempo real.

AR: Exige el uso de **ARKit (iOS)** o **ARCore (Android)**. Aquí la abstracción falla; se necesita acceso directo a la memoria y a los núcleos de procesamiento de visión artificial.



2.2. Parte 2: Análisis Crítico de Aplicaciones (Tinder y Waze)

2.2.1. Ingeniería Inversa de la Decisión

- ✓ **Tinder:** Aunque usa tecnologías híbridas en partes administrativas, su core de **"Swipe" e imágenes** ha priorizado el **Desarrollo Nativo**.
    - **¿Por qué?** Tinder vive de la fluidez del gesto. Un "swipe" con lag rompe la experiencia de usuario (UX). Además, el manejo de memoria para cargar cientos de imágenes de alta calidad mientras se procesa la geolocalización en segundo plano requiere un control granular que solo el desarrollo nativo permite de forma eficiente.
    -

- ✓ **Waze:** Es un aplicativo **100% Nativo** (C++ en el motor de mapas, Kotlin/Swift en la UI).
    - **¿Por qué?** Waze es una aplicación de **cómputo intensivo**. Debe renderizar mapas vectoriales en 3D, calcular rutas en tiempo real mediante algoritmos de grafos y procesar telemetría de GPS cada segundo. Intentar hacer esto en React Native o Flutter generaría un consumo de batería y temperatura excesivos debido a las capas de abstracción.

2.2.2. Análisis de Rendimiento y Arquitectura

- ✓ **Desafíos de Waze:** Su arquitectura debe manejar la **concurrencia** de hilos de red (actualizaciones de tráfico), hilos de renderizado (mapa) e hilos de sensores (GPS/Brújula). El mayor desafío es la **Sincronización de Estado**: que tu posición en el mapa 3D coincida exactamente con la señal de GPS procesada, sin que el renderizado "salte".

- ✓ **Cuello de Botella de Tinder:** Si Tinder usara el "puente" de JS, el cuello de botella sería el **Bridge Traffic**. Al deslizar perfiles rápidamente, el puente se inundaría de eventos (gesto del dedo -> JS -> Lógica -> Native -> Renderizado). Para lograr 60 FPS, Tinder optimiza el renderizado de imágenes mediante librerías como *Glide* o *Fresco* que gestionan el caché directamente en la memoria RAM nativa.

2.2.3. El Desafío de las Nuevas Pantallas

- ✓ **Escenario AR:** Si el futuro son lentes AR, **ambas tecnologías (RN y Flutter) tendrían problemas**, pero Flutter tendría una ligera ventaja al ser un motor de renderizado de píxeles que podría adaptarse más fácilmente a un viewport estereoscópico. Sin embargo, Waze (Nativo) ganaría la carrera, ya que la AR requiere cálculos de **Matriz de Transformación 3D** en tiempo real que se ejecutan mejor sobre C++ (NDK en Android).

2.2.4. Veredicto Crítico: ¿Qué elegir hoy?

Decisión: Desarrollo Nativo. Fundamentación Arquitectónica: 1. **Mapas 3D interactivos:** Los motores de mapas (Google Maps SDK, Mapbox) tienen un rendimiento óptimo cuando se integran directamente con las APIs de metal/vulkan. 2. **Notificaciones de alta frecuencia:** El manejo de Push Notifications con alta carga requiere servicios en segundo plano persistentes. Las capas de abstracción (RN/Flutter) a veces son "matadas" por el sistema operativo para ahorrar batería; el código nativo tiene mayor control sobre el ciclo de vida de la app. 3. **Foldables:** El soporte para pantallas plegables (detección de ángulos, cambio de layout fluido) nace de las APIs nativas de Android (Jetpack WindowManager). Usar un framework cross-platform añade una capa de latencia innecesaria en la transición del estado de la UI.

2.2.5. Pregunta Final: ¿Qué es un aplicativo nativo y cómo se clasifican?

Un aplicativo nativo es aquel desarrollado con el lenguaje de programación y el SDK (Software Development Kit) específico del fabricante del sistema operativo. Esto permite la ejecución de código **"Direct-to-CPU"**.

Clasificación:

1. **Nativo de una sola plataforma (Single-platform):** Escrito en el lenguaje estándar (Swift para iOS, Kotlin para Android). Acceso total y máximo rendimiento.
2. **Nativo Compilado (Cross-platform Compiled):** Como **Flutter**. El código Dart se compila a binario nativo. No usa componentes del SO, pero se ejecuta "nativamente" en el procesador.
3. **Nativo con Puente (Bridge-based):** Como **React Native**. La lógica es JS, pero los elementos de interfaz (botones, inputs) son los mismos que usa una app nativa pura.
4. **Sistemas de bajo nivel (NDK/C++):** Utilizados en Android para partes críticas de apps (como el motor de mapas de Waze o motores de juegos). Es el nivel más cercano al hardware.